Patrones de comportamiento

Hoja de ejercicios (Alumno)

Patrón *Command*

Escenario

Una aplicación de control de una nave espacial debe reaccionar ante distintas situaciones de maniobra. Para ello, dispone de dos subsistemas:

- `ThrusterSystem`, encargado del empuje de la nave.
- `StabilizerSystem`, encargado de la estabilización de la nave.

La lógica de la aplicación debe poder configurar distintas respuestas según la situación detectada. Por ejemplo:

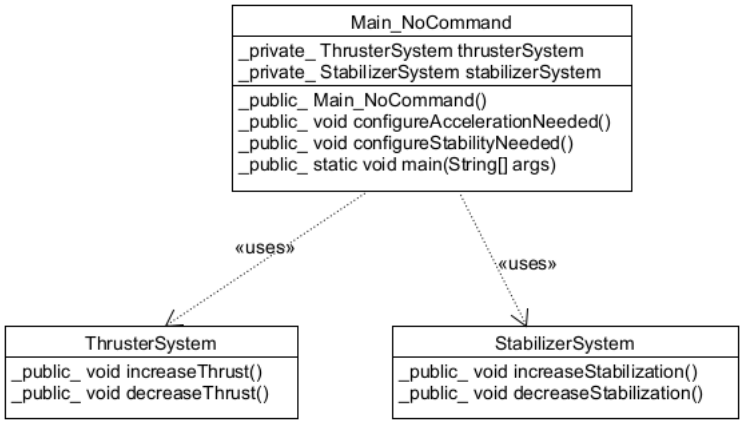
- Si se necesita más aceleración, se debe aumentar el empuje y aumentar la estabilización.
- Si se necesita más estabilidad, se debe reducir el empuje y aumentar la estabilización.

En la versión inicial que se proporciona, la lógica de la aplicación invoca directamente los métodos de los subsistemas, lo que provoca un acoplamiento excesivo entre la decisión de qué hacer y la ejecución concreta de las acciones.

Este proyecto que puedes encontrar en el campus virtual contiene una solución funcional que **no utiliza el patrón *Command***.

UML de la versión sin patrón

A continuación se muestra el diagrama UML de la solución proporcionada.



¿Qué percibimos que no 'funciona como debe'?

En la versión sin patrón:

- La lógica de la aplicación conoce directamente las clases concretas de los subsistemas.
- La lógica de la aplicación invoca directamente métodos como `increaseThrust()`, `decreaseThrust()` o `increaseStabilization()`.
- No se puede razonar realmente al nivel de abstracción de 'órdenes'.
- La aplicación principal tiene mezcladas, por tanto, sus responsbilidades.
- Si se desean añadir nuevas acciones o nuevas combinaciones de acciones, será necesario modificar la lógica principal.

Se pide

Se debe rediseñar el sistema aplicando el patrón *Command*.
La solución deberá incluir, al menos, los siguientes elementos.

Interfaz comando

Crear una interfaz denominada `Command` con el método:

```
execute() : void
```

Invocador

Crear una clase denominada `Actuator` que mantenga una referencia a un objeto de tipo `Command` y proporcione, al menos, los métodos:

```
Actuator(command : Command)
activate() : void
```

Receptores

Deben mantenerse los subsistemas ya existentes: `ThrusterSystem` y `StabilizerSystem` con sus operaciones:

```
increaseThrust() : void
decreaseThrust() : void
increaseStabilization() : void
decreaseStabilization() : void
```

Comandos concretos

Se deben crear, como mínimo, las siguientes clases de comando:

```
IncreaseThrust
DecreaseThrust
IncreaseStabilization
DecreaseStabilization
```

Cada una de estas clases deberá:

- implementar la interfaz `Command`,
- almacenar una referencia al receptor correspondiente,
- e implementar `execute()` delegando la acción en el receptor.

Lógica de la aplicación principal
La aplicación principal deberá:

- crear los receptores,
- crear los objetos comando,
- asociarlos a objetos `Actuator`,
- y ejecutar las acciones mediante llamadas a `activate()`.

La estructura esperada de la aplicación principal debe contener, al menos, los métodos:

```
Main()
configureAccelerationNeeded() : void
configureStabilityNeeded() : void
operate() : void
main(args : String[]) : void
```

Donde:

- Los métodos de configuración de las acciones tienen como objetivo conformar la lista de actuadores con las órdenes descritas en este enunicado para cada escenario.
- El método `operate` sólo debe simular la gestión de las acciones ejecutando todas las que tenga en la lista de actuadores.
- El método principal debe simular que se dispara la ejecución de los dos escenarios para demostrar el funcionamiento correcto de la aplicación.

Objetivo

El objetivo del ejercicio es mostrar cómo el patrón *Command* permite:

- encapsular acciones como objetos,
- desacoplar la lógica de configuración de la ejecución concreta,
- y facilitar la reconfiguración dinámica del comportamiento de la aplicación.